DYNAMIC TRADER · ARCHITECTURE GUIDE

How the whole system fits together

A file-by-file map of the hybrid stack: Rust owns the Nautilus-matching market engine; Go owns research policy, browse UI, and reports. Written for fullstack developers (JS / PHP / Go) who did not write the Rust.

Market OKX BTC-USDT-SWAP **Nautilus** 0.65.0 (verify feature) **Branch** ramtin/cleanup-docs-b567

Stack Rust engine + Go `dt` CLI

---

1. One-minute mental model

OKX zips → Rust tape → scan .bin → Go walk-forward → simulate JSONL → HTML / JSON

Rust = engine

Download archives, build the tick tape, scan millions of RSI×TP/SL combos, batch-simulate fills, optionally verify with NautilusTrader.

Go = product

`dt` ranks windows, runs N/M walk-forward policy, serves the browse UI, writes reports. Changes here compile in seconds.

> **Rule:** change fill rules in Rust (`scan + strategy`) and confirm with `verify` . Change how winners are picked in Go (`internal/walkforward` , `internal/score`).

2. Repository map

```
ppo-trader/
├── Cargo.toml              Rust crate + optional feature verify
├── go.mod / cmd/dt/        Go product CLI entry
├── config/testnet.toml     Fees, paper_only, max leverage
├── scripts/
│   ├── build_dt.sh         macOS-safe Go build (CGO + external linker)
│   ├── price_iter.sh       Fast price-path iteration loop
│   └── md_to_pdf.py        Legacy text PDF (superseded by this HTML)
├── docs/
│   ├── architecture.html   ← this guide (print to PDF)
│   ├── ARCHITECTURE.pdf    Generated from the HTML
│   ├── DT.md               Short CLI cheat sheet
│   └── ARCHITECTURE.md     Markdown twin
├── src/                    Rust library (shared by all engine bins)
├── src/bin/                Rust binaries
└── internal/               Go packages used by dt
```

`data/` is gitignored. Archives and scan outputs live only on your machine.

3. End-to-end data flow

Research path (99% of compute)

1. `download` fetches OKX daily trade zips into `data/okx/BTC-USDT-SWAP/archives/` and verifies ID continuity + volume vs OKX's 1D candle.
2. `Tape::load` turns every trade into Nautilus L1 matching prices with a segment tree for TP/SL `first_hit`.
3. `scan --bucket window` enumerates RSI pairs × TF × TP/SL × leverage into fixed-size `window_*.bin` + `meta.json`.
4. Go `dt walk-forward` ranks those bins, picks finalists, batch-calls Rust `simulate` (JSONL) for closed trades, runs N/M score policy, writes JSON/HTML.

Truth path (slow)

Only the last/best combos go through Nautilus via `verify` / `backtest` / `window_nm_walk_forward --verify`. Same fill rules as the scanner; disagreement fails the run.

4. Rust library (`src/`)

Every library module, what it is for, and roughly how big it is.

FILE	LINES	RESPONSIBILITY
`okx_history.rs`	551	OKX download/verify, Trade/Candle types, known gaps & voided IDs, archive paths
`tape.rs`	383	Tick matching tape + first_hit tree (Nautilus L1 rules without Nautilus)
`scan.rs`	523	Fast RSI-cross + exits simulator; Outcome / ClosedTrade / Record
`tpsl_grid.rs`	772	Vectorized single-TP×SL×lev grid (Entries / Frame / Grid)
`window_score.rs`	849	WindowStats (20-byte records), ScoreModel budget-10 fit
`price_model.rs`	381	OLS candle-return features for price-path research
`exits.rs`	249	Stop / TP ladder math + ExitProfile scale-outs
`filters.rs`	226	EMA trend + ATR filters wrapping crosses
`grid.rs`	185	CLI parsers for lengths/ladders + RSI series helpers
`rsi.rs`	90	Wilder RSI
`rsi_cross.rs`	114	Fast/slow RSI cross signals
`pine_rsi.rs`	146	TradingView “RSI Strategy” port
`fills.rs`	26	Fill / FillKind — no Nautilus dependency
`signals.rs`	20	Signal / BarSignals — no Nautilus dependency
`config.rs`	91	TOML load; rejects live trading & bad leverage
`verify_cmp.rs`	60	Compare scanner vs Nautilus fills
`tradingview.rs`	371	TV WebSocket/HTTP client for tv_check
`strategy.rs`	319	Nautilus ReversalStrategy (feature verify)
`backtest.rs`	259	Nautilus BacktestEngine wrapper (feature verify)
`lib.rs`	21	Module exports; gates backtest/strategy on verify

Key types you will see everywhere

- `okx_history::Trade` / `Candle` — raw market data
- `tape::Tape` — matching prices + `first_hit(from,to,level,up)`
- `exits::Exits` — `{ stop_pct, targets:[(pct,share)], breakeven }`
- `scan::simulate` / `simulate_logged` — fast replay API

- `fills::Fill` — one fill (time, kind, side, size, price, fee)
- `window_score::WindowStats` — per-combo per-window score inputs
- `signals::BarSignals` — trait implemented by RSI / Pine / FilteredCross

---

5. Rust binaries (`src/bin/`)

BINARY	LINES	NEEDS NAUTILUS?	WHAT IT DOES
`download`	52	No	Fetch + verify OKX day archives. Args: `START END` (end exclusive), `--dir`
`scan`	~2.1k	No	`--bucket window|month`. Writes meta.json + .bin records
`simulate`	438	No	Load tape once; JSONL stdin → JSONL stdout batch replay
`verify`	192	Yes	One combo through Nautilus; compare fills JSON
`backtest`	207	Yes	Pine RSI strategy table via Nautilus
`tv_check`	694	No engine	Compare candles/RSI/trades vs TradingView (needs .env cookies)
`window_nm_walk_forward`	4749	Yes (default)	Legacy score path + **active price path** (`--path price`)

`scan/` splits into `main.rs` (bucket switch), `window.rs` (equal windows / staged grid), `month.rs` (calendar months).

---

6. On-disk formats

OKX archives

```
data/okx/BTC-USDT-SWAP/archives/
  BTC-USDT-SWAP-trades-2026-09-01.zip   # or gz
  … plus a verified marker used by is_verified()
```

An archive day runs 16:00 UTC previous day → 16:00 UTC (OKX 1D candle window).

Window scan (`--bucket window`)

- `meta.json` — start/end, timeframes, lengths, TP/SL/leverage grids, window list, fees, size_btc, scan_version (1 dense / 3 staged)
- `window_*.bin` — one **20-byte LE** record per combo:
  `profit f32 · drop f32 · worst f32 · hold_hours f32 · trades u16 · wins u16`
  (`window_score::RECORD = 20`)

Month scan (`--bucket month`)

- `month_YYYY-MM.bin` , `all.bin` — **12-byte LE** records: `sum f32 · drop f32 · trades u32`

---

7. `simulate` JSONL contract

Go walk-forward must batch. Never spawn one process per combo.

Request line

```
{"id":"42","minutes":60,"fast":14,"slow":50,
 "exits":{"stop_pct":1.0,"targets":[[1.0,1.0]],"breakeven":false},
 "leverage":2,"logged":true,"record_fills":false}
```

Response line includes `outcome` (realized, fees, total, ...), optional `closed[]` trades (`exit_ms` , `entry_ms` , `net`), optional `fills[]` .

CLI: `--start --end --config --data --size-btc --balance-usdt --jobs` . Send `{"flush":true}` to force a batch flush.

---

8. Go product (`cmd/dt` + `internal/`)

PACKAGE	LINES	JOB
`cmd/dt`	180	CLI router
`internal/engine`	277	Find/exec Rust bins; Simulate batch client
`internal/browse`	1166	HTTP browser over window/month bins
`internal/walkforward`	982	N/M score-path walk-forward
`internal/score`	459	Budget-10 ScoreModel (Rust parity tests)
`internal/report`	58	Embed HTML + regenerate from JSON

Engine discovery

```
$DT_ENGINE_DIR
./target/relfast
./target/release
./target/debug
$PATH
```

Commands

```
./scripts/build_dt.sh
./dt download|scan|simulate|verify|backtest|tv-check …
./dt browse --scan DIR [--port N] [--layout window|month|auto]
./dt walk-forward --scan DIR [score-path flags…]
./dt report --json PATH
./dt price-walk …     # wraps window_nm_walk_forward --path price
```

---

9. Feature `verify` (Nautilus boundary)

> Without `--features verify` (or with `--no-default-features`), `backtest` and `strategy` modules are not compiled and nautilus crates are not linked. Scanner rebuilds drop from ~45s LTO pain toward ~5–15s relfast.

```
# Fast engine only
cargo build --profile relfast --no-default-features \
  --bin download --bin scan --bin simulate

# Need Nautilus
cargo build --profile relfast --features verify \
  --bin verify --bin backtest --bin window_nm_walk_forward
```

---

10. Config & safety

`config/testnet.toml`

- `risk.paper_only` must be `true` (live rejected)
- `risk.max_leverage` validated (1–10 in config; research often caps lower)
- `venues.okx.maker_fee_bps` / `taker_fee_bps`

Secrets only in `.env` (gitignored). `tv_check` needs TradingView `sessionid` cookies. Never commit keys.

11. Typical workflow

```
# 1) Two years of trades (end exclusive)
cargo run --release --bin download -- 2024-10-31 2026-10-31

# 2) Window scan
cargo run --release --bin scan -- --bucket window \
  --start 2025-10-01 --end 2026-10-01 --hours 24

# 3) Browse
./scripts/build_dt.sh
./dt browse --scan data/windowscan/<dir> --port 8091

# 4) Score walk-forward (Go)
./dt walk-forward --scan data/windowscan/<dir>

# 5) Price path (still Rust)
./dt price-walk --scan data/windowscan/<dir> --verify 0
```

12. What cleanup removed

Deleted as unused after the hybrid split (~7.8k lines):

- `browse_full_scan` / `browse_window_scan` → replaced by `dt browse`
- Thin `full_scan` / `window_scan` wrappers → use `scan --bucket …`
- Legacy orchestration: `rsi_combos`, `walk_forward`, `walk_forward_trades`, `full_walk_forward`, `window_walk_forward` (+ HTML)

Still present: engine bins + `window_nm_walk_forward` for price path.

13. Where to edit what

YOU WANT TO…	EDIT
Change how TP/SL fills	`src/scan.rs + src/strategy.rs, then verify`
Change OKX download / gaps	`src/okx_history.rs`
Change scan grid / staged survivors	`src/bin/scan/window.rs`
Change ranking formula	`src/window_score.rs and internal/score`
Change N/M walk-forward policy	`internal/walkforward/nm.go`
Change browse UI	`internal/browse/`
Change price-path experiment	`src/bin/window_nm_walk_forward.rs + price_model.rs`